

TD Architecture des réseaux — Routage — Corrigé

Objectif de ce corrigé

Ce document reprend le TP "Routage" point par point : rappels, puis correction commentée des trois situations (routage statique simple, routage statique en chaîne, routage dynamique OSPF).

Dans tout ce qui suit, X désigne votre numéro de prise/poste (à remplacer partout par sa valeur réelle). Les exemples utilisent ethX de façon générique : vérifiez toujours les noms réels de vos interfaces avec `ip link` avant de taper une commande, ils peuvent être eth0/eth1/eth2... ou des noms prévisibles type enpXsY.

Avant de commencer : `vm-add debian S1`, `vm-add debian S2`, `vm-add debian R1` (idem pour R2/R3 selon la situation), `vm-ls` pour vérifier, puis `vm-run S1 / vm-run S2` pour démarrer.

Rappels

Adresse IP et masque

Une adresse IPv4 (32 bits, 4 octets décimaux) se découpe en partie réseau (netID) et partie hôte (hostID), la frontière étant fixée par le masque. Exemple avec 192.168.2.200/19 (masque 255.255.224.0) :

```
192.168.2.200    11000000.10101000.00000010.11001000
masque           11111111.11111111.11100000.00000000
ET bit à bit = 192.168.0.0    (adresse réseau)
```

- Adresse réseau = IP ET masque (partie hôte à 0)
- Adresse de diffusion = partie hôte à 1, partie réseau inchangée
- Partie hôte seule = IP ET (NON masque)

Table de routage et décision de routage

Pour chaque paquet, le noyau cherche dans la table de routage la ligne (réseau + masque) qui correspond le mieux à l'IP destination, puis applique un des 4 cas :

1. **Réseau directement connecté** → remise directe sur l'interface locale (ARP puis envoi).
2. **Réseau accessible via un routeur** → le paquet est transmis à ce routeur (next-hop), sans modifier les IP source/destination.
3. **Aucune correspondance mais une route par défaut existe** → envoi au routeur par défaut.
4. **Aucune correspondance et pas de route par défaut** → le paquet n'est pas transmis, un message ICMP Network is unreachable est renvoyé à l'émetteur.

Ce cas 4 est très fréquent en tout début de TP : une machine qui n'a qu'une IP sur son interface ne connaît que son réseau directement connecté, rien d'autre — d'où les `ping` qui échouent tant qu'aucune route (spécifique ou par défaut) n'a été ajoutée.

Statique vs dynamique

- **Routage statique** : les lignes de la table sont saisies à la main (`ip route add ...`), stables tant qu'on ne les change pas, mais ne s'adaptent pas automatiquement à une panne de lien.
- **Routage dynamique** : un protocole (OSPF, RIP, BGP...) fait dialoguer les routeurs entre eux pour construire et mettre à jour la table automatiquement, y compris en cas de coupure d'un lien.

Commandes `ip` utiles

Besoin	Commande
État des interfaces	<code>ip link</code>
Activer une interface	<code>ip link set eth1 up</code>
Voir les adresses IP	<code>ip addr</code>
Ajouter une IP	<code>ip addr add 192.168.1.1/24 dev eth0</code>
Voir le cache ARP (voisinage)	<code>ip neigh</code>
Voir la table de routage	<code>ip route</code>
Ajouter une route	<code>ip route add 172.20.11.0/24 via 192.168.1.253</code>
Supprimer une route	<code>ip route del ...</code>
Route par défaut	<code>ip route add default via <IP passerelle></code>

Partie 2 — Configuration réseau initiale (observation sur S1)

Sur une VM Debian fraîchement créée et démarrée, sans configuration manuelle :

- **Interfaces disponibles** : au moins `lo` (loopback) + les interfaces réseau attribuées à la VM (souvent plusieurs `ethX`, même si non utilisées dans cette première étape). `ip link` les liste toutes.
- **IP** : en général aucune, sauf `lo` en 127.0.0.1/8 — `ip addr` le confirme.
- **Routes** : table vide ou quasi vide (juste la route locale du loopback) tant qu'aucune IP n'a été ajoutée sur une interface "utile" — `ip route`.

- **Type de liens** : `ip link` indique `ether` (Ethernet) pour les interfaces physiques/virtuelles, `loopback` pour `lo`.
- **Adresses MAC** : visibles dans la sortie de `ip link` (champ `link/ether`), une par interface.
- **Cache ARP** : vide au départ (`ip neigh` ne renvoie rien tant qu'aucune communication n'a eu lieu sur le réseau local).

Après `ip addr add 192.168.X.101/24 dev eth3` sur S1 et `ip addr add 192.168.X.102/24 dev eth3` sur S2 : une route directement connectée vers `192.168.X.0/24` apparaît automatiquement dans `ip route` sur chaque machine (ajoutée par le noyau, pas besoin de `ip route add`). Le `ping` entre les deux passe alors (même réseau, remise directe — cas 1 du rappel). Après le `ping`, `ip neigh` sur S1 affiche l'entrée ARP de S2 (et réciproquement), remplie par la résolution ARP déclenchée par le `ping`. `vm-tcpdump` sur l'une des VM pendant le `ping` montre les paquets `ICMP echo request/echo reply` (et la résolution ARP juste avant, la première fois).

Situation 1 (S1 — R1 — S2, 2 réseaux)

Topologie : S1 (172.X.1.1) — [172.X.1.0/24] — R1 (172.X.1.254 / 172.X.2.254) — [172.X.2.0/24] — S2 (172.X.2.1). 4 interfaces au total : 1 sur S1, 2 sur R1, 1 sur S2.

Configuration des IP (sans route pour l'instant)

```
# S1
ip addr add 172.X.1.1/24 dev eth1

# R1 (deux interfaces homologues de celles de S1 et S2)
ip addr add 172.X.1.254/24 dev eth1
ip addr add 172.X.2.254/24 dev eth2

# S2
ip addr add 172.X.2.1/24 dev eth1
```

`ip route` sur chaque machine ne montre alors que la route directement connectée à son propre réseau (ajoutée automatiquement).

Ping depuis S1 vers les 4 interfaces — ce qui marche et pourquoi

Destination	Résultat	Explication
172.X.1.1 (soi-même)	OK	adresse locale
172.X.1.254 (R1, interface proche)	OK	même réseau 172.X.1.0/24 → remise directe (cas 1)
172.X.2.254 (R1, interface lointaine)	échec	S1 ne connaît que 172.X.1.0/24 ; 172.X.2.0/24 est absent de sa table et il n'y a pas de route par défaut → Network

unreachable (cas 4)

172.X.2.1 (S2) **échec** même raison que ci-dessus

À ce stade, le fait que R1 relie physiquement les deux réseaux ne suffit pas : S1 n'a tout simplement pas l'information "pour joindre 172.X.2.0/24, il faut passer par 172.X.1.254".

Ajout de la route sur S1

```
ip route add 172.X.2.0/24 via 172.X.1.254
```

Cette fois, un ping de S1 vers R1 (172.X.2.254) fonctionne déjà : R1 reçoit le paquet sur eth1, mais la destination est une de ses propres adresses locales (172.X.2.254) — la remise se fait au niveau de la pile IP de R1 elle-même, **sans avoir besoin du forwarding**. En revanche, un ping de S1 vers S2 (172.X.2.1) échoue encore : cette fois R1 doit *retransmettre* un paquet qui n'est pas destiné à une de ses propres adresses, ce qui nécessite l'IP forwarding.

Activer le forwarding sur R1

```
cat /proc/sys/net/ipv4/ip_forward      # vaut 0 par défaut
echo 1 > /proc/sys/net/ipv4/ip_forward # ou : sysctl net.ipv4.ip_forward=1
```

echo 1 > /proc/sys/net/ipv4/ip_forward écrit la valeur 1 dans le pseudo-fichier /proc/sys/net/ipv4/ip_forward, qui est l'interface exposée par le noyau (procfs) vers le paramètre ip_forward : cela active immédiatement la retransmission de paquets IP entre interfaces, mais **seulement pour la session en cours** (perdu au redémarrage). Pour rendre le changement permanent : décommenter net.ipv4.ip_forward=1 dans /etc/sysctl.conf, puis sysctl -p pour recharger la configuration depuis ce fichier.

Route retour sur S2, puis test

```
ip route add 172.X.1.0/24 via 172.X.2.254
```

Un ping S1 ↔ S2 fonctionne alors dans les deux sens : S1 sait joindre 172.X.2.0/24 via R1, S2 sait joindre 172.X.1.0/24 via R1, et R1 forward.

Retour à des routes par défaut uniquement

```
# Sur S1
ip route del 172.X.2.0/24 via 172.X.1.254
ip route add default via 172.X.1.254
```

```
# Sur S2
ip route del 172.X.1.0/24 via 172.X.2.254
ip route add default via 172.X.2.254
```

Le ping continue de fonctionner : une route par défaut couvre *tout* réseau non directement connu (cas 3 du rappel), donc elle remplace ici sans problème la route spécifique vers 172.X.2.0/24 (et inversement) — c'est d'ailleurs la configuration "normale" d'une station qui n'est pas elle-même un routeur, comme le rappelle l'énoncé.

Situation 2 (S1 — R1 — R2 — R3 — S3, chaîne linéaire)

Topologie : trois routeurs en ligne, chacun avec sa station.

- S1 (172.X.1.1) — R1 (172.X.1.254 / 10.X.1.1)
- R1 (10.X.1.1) — [10.X.1.0/24] — R2 (10.X.1.2)
- S2 (172.X.2.1) — R2 (172.X.2.254)
- R2 (10.X.2.2) — [10.X.2.0/24] — R3 (10.X.2.3)
- S3 (172.X.3.1) — R3 (172.X.3.254 / 10.X.2.3)

Adressage

```
# S1                                     # S2                                     # S3
ip addr add 172.X.1.1/24 dev eth1 ip addr add 172.X.2.1/24 dev eth1 ip addr
add 172.X.3.1/24 dev eth1
ip route add default via 172.X.1.254   via 172.X.2.254               via
172.X.3.254

# R1                                     # R2                                     # R3
ip addr add 172.X.1.254/24 dev eth1 ip addr add 172.X.2.254/24 dev eth2 ip
addr add 172.X.3.254/24 dev eth3
ip addr add 10.X.1.1/24 dev eth2      ip addr add 10.X.1.2/24 dev eth1      ip
addr add 10.X.2.3/24 dev eth1
                                     ip addr add 10.X.2.2/24 dev eth3
```

(adaptez les noms d'ethX à ce que renvoie `ip link` sur chaque VM).

Routing — pourquoi une seule route par défaut ne suffit pas partout

R1 et R3 n'ont qu'une seule direction possible vers le reste du réseau : une route par défaut leur suffit.

```
# R1
ip route add default via 10.X.1.2 # tout ce qui n'est pas 172.X.1.0/24 ou
10.X.1.0/24 part vers R2

# R3
ip route add default via 10.X.2.2 # tout ce qui n'est pas 172.X.3.0/24 ou
10.X.2.0/24 part vers R2
```

R2, lui, a **deux** directions distinctes (vers R1 et vers R3) : une route par défaut unique ne peut pas exprimer "selon la destination, sortir par l'un ou l'autre". Il lui faut deux routes spécifiques :

```
# R2
ip route add 172.X.1.0/24 via 10.X.1.1 # vers S1, en passant par R1
ip route add 172.X.3.0/24 via 10.X.2.3 # vers S3, en passant par R3
```

Forwarding et test

```
sysctl -w net.ipv4.ip_forward=1 # sur R1, R2 et R3
```

Un ping de S1 vers S3 doit alors traverser R1 → R2 → R3. `traceroute -n <IP de S3>` depuis S1 permet de vérifier le chemin exact (les trois sauts intermédiaires 10.X.1.1, 10.X.2.2/10.X.1.2 selon le sens, puis l'arrivée).

Routage dynamique — OSPF (R1, R2, R3 en triangle complet)

Topologie : R1, R2 et R3 sont reliés deux à deux (maillage complet), chacun avec sa propre station :

- S1 — R1 sur 172.X.1.0/24, S2 — R2 sur 172.X.2.0/24, S3 — R3 sur 172.X.3.0/24
- R1 (10.X.3.1) — R2 (10.X.3.2) sur 10.X.3.0/24
- R2 (10.X.1.2) — R3 (10.X.1.3) sur 10.X.1.0/24
- R1 (10.X.2.1) — R3 (10.X.2.3) sur 10.X.2.0/24

Ce maillage complet est précisément ce qui permet à OSPF de recalculer un chemin alternatif si un lien tombe (routeur "du milieu" de secours toujours disponible).

Stations : IP + route par défaut, rien d'autre

```
ip addr add 172.X.1.1/24 dev eth1 # sur S1 (adapter pour S2, S3)
ip route add default via 172.X.1.254
```

Seuls **R1, R2, R3** seront ensuite modifiés pour le routage — les stations n'ont besoin que de leur route par défaut.

Installation de frr

```
apt update
apt install frr
```

frr (fork de **quagga**, lui-même fork de **zebra**) implémente les protocoles de routage dynamique usuels (OSPF, RIP, BGP...) avec un démon par protocole (**ospfd** pour OSPF) coordonné par le démon central **zebra**. **frr** prend les décisions de routage ; c'est ensuite le noyau Linux qui se charge du transfert effectif des paquets. **vttysh** est le terminal de consultation/configuration à chaud, façon routeur Cisco.

/etc/frr/daemons — activer ospfd

```
ospfd=yes
```

/etc/frr/frr.conf — exemple pour R1

(A = numéro du routeur configuré, ici 1 ; B, C = les deux autres, ici 2 et 3 — à adapter réellement sur R2 et R3 avec leurs propres interfaces/IP.)

```
interface eth1
 ip address 172.X.1.254/24
!
interface eth2
 ip address 10.X.3.1/24
 bandwidth 100000
!
interface eth3
 ip address 10.X.2.1/24
 bandwidth 100000
!
router ospf
```

```
network 172.X.1.0/24 area 0
network 10.X.3.0/24 area 0
network 10.X.2.0/24 area 0
!
```

bandwidth <valeur> (en kbit/s) sert à donner à OSPF une bande passante réaliste par interface, car OSPF calcule le **coût** d'une route à partir de la bande passante (plus elle est élevée, plus le coût est faible) — sans cela, des liens de débits très différents pourraient être traités comme équivalents. **network ... area 0** déclare sur quels réseaux OSPF doit chercher des voisins et annoncer des routes ; ici toutes les interfaces de R1 sont dans l'aire 0 (aire unique, cas le plus simple).

Droits, logs, démarrage

```
chown frr:frr /etc/frr/frr.conf && chmod 640 /etc/frr/frr.conf
mkdir -p /var/log/frr && chown frr:frr /var/log/frr
systemctl restart frr
ps aux | grep -E 'zebra|ospfd' # vérifier que les démons tournent
```

Lire la table de routage

```
vtysh -c "show ip route"
```

```
0 10.1.2.0/24 [110/1] is directly connected, interface, 00:02:30
```

Le [110/1] se lit : 110 = distance administrative d'OSPF (priorité entre sources de routes : 0 connecté, 1 statique, 20 BGP, 110 OSPF, 120 RIP — plus la valeur est basse, plus la route est préférée en cas de conflit entre protocoles), 1 = coût de la route calculé par OSPF (fonction de la bande passante cumulée du chemin). La même information est visible côté noyau avec **ip route** (les routes apprises par OSPF y apparaissent comme n'importe quelle autre route, avec **proto zebra** ou **proto ospf** selon la version).

Côté état du protocole lui-même : **vtysh** puis **show ip ospf**, **show ip ospf neighbor** (voisins OSPF découverts et leur état, **Full** = adjacence complète), **show ip ospf route** (table calculée par OSPF avant injection dans le noyau).

Test et simulation de panne

```
ping <IP de S2> # depuis S1
traceroute -n <IP de S2> # -I pour forcer ICMP si besoin
```

Pour simuler la coupure du lien R1–R2 : soit couper l'interface concernée (**ip link set eth2 down** sur R1, équivalent logiciel d'un débranchement de câble), soit bloquer le trafic OSPF avec **iptables** (par exemple en droppant le protocole OSPF, numéro IP 89, sur l'interface visée) sans couper le lien physiquement.

Après la coupure, le **ping/traceroute** S1 ↔ S2 doit continuer à fonctionner : grâce au maillage complet, OSPF détecte la perte de voisinage sur R1–R2 (le voisin disparaît de **show ip ospf neighbor**), recalcule les routes en passant par R3, et le trafic emprunte désormais R1 → R3 → R2 (visible dans **traceroute** et dans la nouvelle table **show ip route**).